

Ejercicios: Análisis Amortizado

Análisis y Diseño de Algoritmos

7 de junio de 2026

Ejercicio 1. Supongamos que realizamos una secuencia de operaciones (PUSH y POP) sobre una pila cuyo tamaño nunca excede k . Después de cada k operaciones, hacemos una copia completa de la pila con fines de respaldo. Demuestre que el costo de n operaciones de pila, incluyendo las copias de respaldo, es $O(n)$, asignando costos amortizados adecuados a las distintas operaciones de la pila.

Ejercicio 2. Suponga que tenemos un algoritmo $\text{OPERACION}(T, k)$ que recibe un árbol binario completo T y un nodo k de dicho árbol y realiza un número de pasos proporcional al número de descendientes de k (recuerde que un nodo es descendiente de sí mismo). Dado un árbol binario completo T con conjunto de nodos $1, \dots, n$, invocamos a $\text{OPERACION}(T, k)$ n veces consecutivas, una para cada $k = 1 \dots n$.

¿Cual es el costo amortizado, en notación O-grande, de cada llamada a OPERACION ? Justifique adecuadamente.

Ejercicio 3. Una secuencia de n operaciones es ejecutada sobre cierta estructura de datos, de manera tal que la operación cuesta i , cuando i es una potencia de 2, y 1 en caso contrario.

¿Cual es el costo amortizado, en notación O-grande, de cada operación? Justifique adecuadamente **usando el método de recargas**.

Ejercicio 4. Considere una tabla dinámica que duplica su capacidad cuando está llena y además, cuando el número de elementos almacenados cae por debajo de un cuarto de la capacidad, la capacidad se reduce a la mitad. La tabla soporta las siguientes operaciones:

- **Insert(x):** inserta el elemento x en la tabla.
 - **Delete():** elimina un elemento de la tabla.
- a) Muestre que una estrategia ingenua que reduce la capacidad a la mitad cada vez que la tabla llega a estar medio vacía puede llevar a un costo amortizado no constante.
- b) Analice la estrategia que reduce la capacidad a la mitad solo cuando el número de elementos almacenados es menor que un cuarto de la capacidad.
- c) Demuestre que tanto **Insert** como **Delete** tienen costo amortizado $O(1)$ bajo esta estrategia.

Ejercicio 5. Una secuencia de n operaciones es ejecutada sobre cierta estructura de datos, de manera tal que la operación cuesta i , cuando i es una potencia de 3, y 2 en caso contrario.

¿Cual es el costo amortizado, en notación O-grande, de cada operación? Justifique adecuadamente **usando el método de recargas**.

Ejercicio 6. Una secuencia de n operaciones es ejecutada sobre cierta estructura de datos, de manera tal que la operación cuesta i , cuando i es un cuadrado perfecto, y 2 en caso contrario.

¿Cual es el costo amortizado, en notación O-grande, de cada operación? Puede usar el método agregado o de recargas. En cualquier caso debe justificar adecuadamente.

Ejercicio 7. Un *arreglo extensible* (*extendable array*) es una estructura de datos que almacena una secuencia de elementos y soporta las siguientes operaciones:

- **AddToFront(x)**: agrega x al comienzo de la secuencia.
- **AddToEnd(x)**: agrega x al final de la secuencia.
- **Lookup(k)**: devuelve el k -ésimo elemento de la secuencia, o **Null** si la longitud actual de la secuencia es menor que k .

Describa una estructura de datos simple que implemente un arreglo extensible. Sus algoritmos **AddToFront** y **AddToEnd** deben ejecutarse en tiempo amortizado $O(1)$, y su algoritmo **Lookup** debe ejecutarse en tiempo de peor caso $O(1)$. La estructura de datos debe usar $O(n)$ espacio, donde n es la longitud actual de la secuencia.